

AI Engineer Intern Assignment

Duration: 2 Hours

Objective

Evaluate the candidate's ability to build a small retrieval-based question answering (RAG) system in Python and to test whether it actually works.

Problem Statement

A logistics company has an internal handbook. Support staff waste time searching it manually and sometimes quote the wrong policy.

Build a simple tool that:

Question → Search the documents → Send relevant text to an LLM → Answer with citation

The tool must also say "I don't know" when the handbook does not contain the answer.

Dataset

- A corpus/ folder is provided with 16 short Markdown documents (fuel charges, claim rules, facility hours, escalation policy, etc.).
- A questions.json file is also provided with 8 sample questions and expected answers. One of them cannot be answered from the documents.

(The company is fictional, so the model cannot answer from memory — it must actually search the documents.)

Tasks

Task 1: Load and Index the Documents (20 minutes)

- Read all .md files from corpus/
- Split each document into smaller chunks (any reasonable size)
- Build a simple search index using TF-IDF or BM25 (sklearn or rank_bm25 — no embedding API needed)
- Keep track of which document each chunk came from

Task 2: Retrieval (20 minutes)

- Write a function that takes a question and returns the top 3–5 relevant chunks
- Print the chunks and their scores for 2–3 test questions
- Check by eye: is the correct document actually coming up?

Task 3: Answer Generation (30 minutes)

- Write a function:

```
def answer(question: str) -> dict:
```

```
    return {  
        "answer": "...",      # the answer text  
        "citations": ["doc-name"], # which file(s) it came  
        "supported": True     # False if not found in documents    }  
    }
```

Requirements:

- Pass only the retrieved chunks to the LLM, not the whole corpus
- The answer must be based only on the retrieved text
- If the documents do not contain the answer, set supported = False and reply that the handbook does not cover it
- Citations must be real filenames from corpus/

Task 4: Testing and Analysis (25 minutes)

- Run all 8 sample questions through your answer() function
- Compare against the expected answers and report how many were correct
- Check the unanswerable question — did your system correctly refuse?
- Print 2–3 example outputs (question, answer, citation)
- Write 3–4 lines on what went wrong and why

Task 5: Conceptual Questions (10 minutes)

Short answers (2–3 lines each):

- Why do we split documents into chunks instead of sending all 16 documents to the model?
- Why is it important for the system to refuse to answer some questions?
- What is one weakness of keyword search (TF-IDF/BM25)? How would embeddings help?
- What one improvement would you make if given more time?

(15 minutes are left as buffer for setup and debugging.)

Bonus (Optional)

- Use embeddings (e.g. sentence-transformers) instead of keyword search
- Print retrieval scores along with the answer
- Handle the case where two documents give conflicting information
- Add a small UI using Streamlit or Gradio

Deliverables

- Python script or Jupyter Notebook
- Clean, readable code with comments on the key parts

- A brief explanation (3–5 lines) of your approach and the choices you made
- Your test results from Task 4

Constraints

- Use Python only
- Answers must come from the provided documents, not the model's own knowledge
- Every answer must include a citation
- The system must be able to say "I don't know"
- Keep it lightweight and runnable in a short time no database or deployment needed